



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

**РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К ПРЕДДИПЛОМНОЙ ПРАКТИКЕ
НА ТЕМУ:**

« Компилятор базисного подмножества

Рефала-5 с представлением данных,

использующим раскрученные очереди »

Студент _____
(Группа)

(Подпись, дата)

(И.О. Фамилия)

Руководитель

(Подпись, дата)

(И.О. Фамилия)

2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Технологическая часть	5
1.1 Инструкция пользователя	5
1.1.1 Требования к среде	5
1.1.2 Сборка	5
1.1.3 Запуск интерпретатора	6
1.1.4 Запуск тестов	6
1.2 Структура данных: раскрученная двусторонняя очередь . .	7
1.2.1 Sealed interface Expr<T>	7
1.2.2 Параметры буферов	8
1.2.3 ShallowDeque<T>: мелкий буфер	8
1.2.4 DeepDeque<T>: глубокий уровень	9
1.2.5 Конкатенация: четыре случая	10
1.3 Цикл нормализации	11
1.3.1 Поле зрения и иерархия ViewTerm	11
1.3.2 Двустековая реализация нормализатора	12
1.3.3 Оптимизация BulkPassive	13
2 Аналитическая часть	15
2.1 Стратегия тестирования	15
2.2 Состав тестов	15
2.2.1 Юнит-тесты раскрученной очереди	15
2.2.2 Юнит-тесты лексера, парсера и семантики	16
2.2.3 Юнит-тесты исполнения	17
2.2.4 Интеграционные тесты	17
2.2.5 Совместимостные тесты (AutotestsRunnerTest)	17
2.3 Покрытие кода	18
2.4 Сравнительные прогоны	18
2.4.1 Условия измерений	18
2.4.2 Тестовая программа Loop/Select	19
2.4.3 Методика измерений	19

2.4.4	Результаты	20
2.4.5	Обсуждение результатов	21
	ЗАКЛЮЧЕНИЕ	23
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	24
A	Исходный код бенчмарка Loop/Select	25
Б	Инструкция по воспроизведению измерений	26
В	Реализация функций конкатенации	28

ВВЕДЕНИЕ

Рефал-5 — язык программирования, в основе которого лежит переписывание объектных выражений по набору правил. Программа на Рефале задаётся как функции, состоящие из предложений вида «образец $\rightarrow$ результат»; интерпретатор сопоставляет аргументы вызова с образцами и собирает правую часть подошедшего предложения. Скорость такой машины определяется не только тем, сколько шагов нормализации она делает, но и тем, как устроены сами операции над объектными выражениями: вставка с концов, извлечение, конкатенация.

Цель производственной практики — довести до работающего состояния реализацию базисного подмножества Рефала-5 на Java, в которой объектное выражение представлено раскрученной (bootstrapped) двусторонней очередью по Окасаки. От этой структуры ожидается амортизированно константная конкатенация, и на её фоне реализация должна обогнать классические списочные интерпретаторы Рефала-5 на программах, где такая конкатенация встречается на каждом шаге нормализации.

Конкретные задачи, поставленные на практику:

- реализовать структуру данных Expr (мелкий и глубокий уровни, конкатенация по четырём случаям) и итеративный цикл нормализации с двумя стеками;
- написать тестовый каркас, в который входят юнит-тесты, интеграционные проверки и прогон чужих наборов автотестов;
- оформить инструкцию пользователя: сборка, запуск, опции командной строки, коды возврата;
- провести сравнительный замер с Refal-5 PZ и Refal-5J на программе, где списочное представление даёт квадратичную сложность, и подтвердить или опровергнуть теоретическое преимущество.

Теоретическое обоснование выбора структуры данных (мелкий и глубокий уровни, инварианты, ребалансировка, амортизированные оценки) сделано в рамках научно-исследовательской работы и в отчёте по практике не повторяется. Здесь акцент смещён на инженерную сторону: что именно реализовано, как оно тестируется и какие числа даёт на стенде.

1 Технологическая часть

1.1 Инструкция пользователя

1.1.1 Требования к среде

Для сборки и запуска интерпретатора необходимо следующее программное обеспечение (таблица 1). Проверка наличия и версий выполняется командами `java --version`, `mvn --version` в терминале.

Таблица 1 — Требования к среде разработки

Компонент	Минимальная версия	Примечание
Java Development Kit	25 LTS	Сборка и запуск
Apache Maven	3.9.x	Сборка проекта
Python 3	3.9+	Только для построения графиков

1.1.2 Сборка

Проект собирается командой Apache Maven из корня репозитория `refal5j-impl/`:

Listing 1: Команда сборки

```
1 mvn package -DskipTests
```

Результат сборки — файл `target/refal5q.jar`, самодостаточный JAR-архив с прописанным классом `Main-Class: refal.Main`. Все зависимости рантайма включены в него через плагин `Maven Shade`, поэтому для запуска достаточно стандартной JVM без дополнительных библиотек.

Сборка с полным прогоном тестов выполняется командой `mvn package` (без флага `-DskipTests`) или `mvn verify`, которая дополнительно формирует отчёт о покрытии кода инструментом `JaCoCo`.

1.1.3 Запуск интерпретатора

Интерпретатор запускается через стандартную команду `java -jar`:

Listing 2: Форма запуска

```
1 java -jar target/refal5q.jar [OPTIONS] program.ref
```

Позиционный аргумент — путь к исходному файлу `.ref`. Поддерживаемые опции перечислены в таблице 2.

Таблица 2 — Опции командной строки

Опция	Описание
<code>-entry <имя></code>	Явное указание входной функции; имеет приоритет над <code>\$ENTRY</code> в файле
<code>-step-limit <N></code>	Максимальное число шагов нормализации (по умолчанию — 100 000)
<code>-expr <терм...></code>	Начальное выражение, передаваемое входной функции вместо пустого

Код завершения процесса отражает результат выполнения (таблица 3).

Таблица 3 — Коды завершения процесса

Код	Условие
0	Успешное завершение или <code><Exit 0></code>
1	Ошибка лексического, синтаксического или семантического анализа
2	Ошибка времени выполнения: нет подходящего правила, превышение лимита шагов
n	Значение n , переданное встроенной функции <code><Exit n></code>

Пример запуска программы `hello.ref` с явным указанием точки входа:

Listing 3: Пример запуска

```
1 java -jar target/refal5q.jar --entry Go hello.ref
```

1.1.4 Запуск тестов

Полный набор тестов запускается одной командой Maven:

Listing 4: Запуск тестов

```
1 mvn test
2 mvn verify
3 mvn test -Dtest=AutotestsRunnerTest
```

Первые две команды прогоняют 143 JUnit-теста, покрывающих все компоненты проекта. Третья команда запускает отдельный класс `AutotestsRunnerTest`, который вызывает интерпретатор на 81 внешней программе из двух наборов: авто-тесты репозитория `refal-5j` и десахаренные тесты репозитория `refal-5-framework`. Каждая программа выполняется в отдельном процессе с тайм-аутом 20 секунд.

Отчёт JaCoCo формируется в каталоге `target/site/jacoco/`; открыть его можно браузером, указав путь к `index.html`.

1.2 Структура данных: раскрученная двусторонняя очередь

Объектное выражение в нашей реализации хранится в раскрученной (bootstrapped) двусторонней очереди по Окасаки [1]. У неё амортизированно константная конкатенация — та самая операция, на которой классические списочные интерпретаторы Рефала (включая `Refal-5 PZ` и `Refal-5J`) тратят линейное время.

1.2.1 Sealed interface `Expr<T>`

Контейнер выражения объявлен как обобщённый запечатанный интерфейс с двумя реализациями:

Listing 5: Запечатанный интерфейс `Expr<T>`

```
1 public sealed interface Expr<T> extends Iterable<T>
2     permits ShallowDeque, DeepDeque {
3     boolean isEmpty();
4     int size();
5     PeekResult<T> separateFront();
6     PeekResult<T> separateBack();
```

```

7   Expr<T> pushFront(T x);
8   Expr<T> pushBack(T x);
9   static <T> Expr<T> empty()          { return ShallowDeque.empty(); }
10  static <T> Expr<T> singleton(T x) { return
    ↪ ShallowDeque.<T>empty().pushBack(x); }
11  static <T> Expr<T> concat(Expr<T> a, Expr<T> b) { return
    ↪ ExprOps.concat(a, b); }
12 }

```

Слово `sealed` запрещает любые другие наследники. Компилятор Java 25 на этом основании проверяет полноту разбора в `switch`-выражениях, поэтому в коде конкатенации мы можем перечислить ровно четыре случая по типам операндов и не писать ветку `default`: её отсутствие гарантировано системой типов.

1.2.2 Параметры буферов

Ёмкость мелкого буфера — константа `ShallowDeque.CAPACITY = 9`, нижняя граница заполненности `MIN_FILL = 5`, то есть $\lceil CAPACITY/2 \rceil$. Нечётное значение выбрано по арифметическим соображениям: при переполнении мелкого буфера суммарно получается $CAPACITY + 1 = 10$ элементов; они делятся ровно пополам на два фрагмента по 5, и оба фрагмента сразу удовлетворяют `MIN_FILL`. С чётной ёмкостью пришлось бы либо мириться с остатком в один элемент, либо добавлять отдельную обработку этого случая — ни то ни другое не даёт выигрыша по свойствам структуры.

На практике глубина рекурсии `DeepDeque` при таком `CAPACITY` почти не выходит за два уровня: в тестовых программах с выражениями длиной до десятков тысяч термов средняя очередь сама ни разу не успевает стать глубокой.

1.2.3 `ShallowDeque<T>`: мелкий буфер

`ShallowDeque<T>` — иммутабельный класс с тремя полями: массив фиксированной длины `Object[] buf` размером `CAPACITY`, индекс головы `head` и текущий размер `sz`. Доступ по логическому индексу i выполняется через единственную формулу:

Listing 6: Циклическое индексирование в ShallowDeque

```
1 int idx(int i) {  
2     return Math.floorMod(head + i, CAPACITY);  
3 }
```

`floorMod` здесь стоит вместо обычного `%`, потому что в Java оператор `%` при отрицательном делимом возвращает отрицательный остаток, а нам нужен индекс в массиве. `pushFrontTight` как раз сдвигает голову на -1 , и без `floorMod` в этой точке пришлось бы вручную добавлять `CAPACITY`.

В классе определены два варианта операций добавления:

- `pushFrontTight(T)` и `pushBackTight(T)` — внутренние, возвращают `ShallowDeque<T>` и бросают `IllegalStateException` при переполнении; применяются там, где вызывающий код гарантирует наличие свободного места (например, при формировании буферов во время конкатенации);
- `pushFront(T)` и `pushBack(T)` — публичные (методы интерфейса `Expr<T>`); при переполнении автоматически переходят в глубокий уровень через `DequeDeque.mkDeep`.

1.2.4 DequeDeque<T>: глубокий уровень

`DequeDeque<T>` реализует тройку $\langle front, mid, back \rangle$:

Listing 7: Поля DequeDeque<T>

```
1 public final class DequeDeque<T> implements Expr<T> {  
2     private final ShallowDeque<T> front;  
3     private final Expr<ShallowDeque<T>> mid;  
4     private final ShallowDeque<T> back;  
5     private final int sz;  
6     ...  
7 }
```

Тип поля `mid` — `Expr<ShallowDeque<T>`, и именно здесь проявляется «раскрутка»: средняя очередь хранит мелкие буферы, но сама она тоже `Expr<...>`, то

есть может оказаться как `ShallowDeque`, так и `DeepDeque` следующего уровня. На разумных размерах выражений рекурсия редко уходит глубже второго уровня.

Конструктор `mkDeep` автоматически схлопывает глубокую очередь обратно в мелкую, когда суммарное число элементов не превышает `CAPACITY`:

Listing 8: Конструктор `mkDeep` с откатом в `ShallowDeque`

```
1 static <T> Expr<T> mkDeep(ShallowDeque<T> front,
2                             Expr<ShallowDeque<T>> mid,
3                             ShallowDeque<T> back, int total) {
4     if (total <= CAPACITY) {
5         ShallowDeque<T> flat = ShallowDeque.empty();
6         for (T x : iterableOf(front)) flat = flat.pushBackTight(x);
7         for (ShallowDeque<T> bucket : iterableOf(mid))
8             for (T x : iterableOf(bucket)) flat = flat.pushBackTight(x);
9         for (T x : iterableOf(back)) flat = flat.pushBackTight(x);
10        return flat;
11    }
12    return new DeepDeque<>(front, mid, back, total);
13 }
```

Параметр `total` передаётся явно, чтобы не пересчитывать его внутри: `mid.size()` возвращает число буферов в средней очереди, а не суммарное число листовых элементов, поэтому наивная формула `front.size() + mid.size() + back.size()` оказалась бы некорректной. Каждый вызов `mkDeep` обновляет `total` инкрементально — прибавляя или вычитая единицу по ходу операции добавления или извлечения.

1.2.5 Конкатенация: четыре случая

Конкатенация реализована в служебном классе `ExprOps` с разбором четырёх случаев по мелкости/глубине операндов:

Listing 9: Разбор четырёх случаев конкатенации

```
1 static <T> Expr<T> concat(Expr<T> a, Expr<T> b) {
2     if (a.isEmpty()) return b;
3     if (b.isEmpty()) return a;
```

```

4   if (a instanceof ShallowDeque<T> sa && b instanceof ShallowDeque<T>
    ↪ sb)
5       return concatSS(sa, sb);
6   if (a instanceof ShallowDeque<T> sa && b instanceof DeepDeque<T> db)
7       return concatSD(sa, db);
8   if (a instanceof DeepDeque<T> da && b instanceof ShallowDeque<T> sb)
9       return concatDS(da, sb);
10  DeepDeque<T> da = (DeepDeque<T>) a;
11  DeepDeque<T> db = (DeepDeque<T>) b;
12  return concatDD(da, db);
13 }

```

В случае Деер + Деер пограничные буферы `a.back` и `b.front` склеиваются в один буфер, а `concat` вызывается рекурсивно уже на средних очередях. На каждом уровне рекурсии работа ограничена константой, а сама рекурсия идёт не глубже логарифма от длины выражения — так Окасаки и получает амортизированную константу [1].

1.3 Цикл нормализации

1.3.1 Поле зрения и иерархия ViewTerm

Нормализация выражения выполняется итеративным проходом по полю зрения — линейной последовательности элементарных единиц. Каждая такая единица описывается запечатанным интерфейсом `ViewTerm`:

Listing 10: Иерархия ViewTerm

```

1  public sealed interface ViewTerm
2      permits ViewTerm.Passive, ViewTerm.BulkPassive,
3              ViewTerm.AngOpen, ViewTerm.AngClose,
4              ViewTerm.ParenOpen, ViewTerm.ParenClose {
5
6      record Passive(Term term) implements ViewTerm {}
7      record BulkPassive(Expr<Term> items) implements ViewTerm {}
8      record AngOpen(String fn) implements ViewTerm {}
9      record AngClose() implements ViewTerm {}
10     record ParenOpen() implements ViewTerm {}

```

```

11     record ParenClose()                                implements ViewTerm {}
12 }

```

Термы-атомы (Sym, Num, Str) и скобочные термы Br после нормализации вложенных выражений представляются как Passive. Вызов <f args> разворачивается в последовательность AngOpen(f), элементы аргументов, AngClose. Скобочный терм (e.X) аналогично разворачивается через ParenOpen и ParenClose. BulkPassive отвечает за оптимизацию связывания переменных и описан в следующем подразделе.

1.3.2 Двустековая реализация нормализатора

Нормализация работает с двумя стеками — left и right:

- right содержит ещё не обработанные элементы поля зрения;
- left накапливает уже пассивные элементы — те, в которых не осталось ни одного активного вызова.

Исходное выражение сначала сплющивается в right слева-направо. Затем главный цикл извлекает элементы из right по одному с головы. Пассивные элементы (Passive, BulkPassive) и открывающие маркеры (AngOpen, ParenOpen) просто перекладываются в left. Обработка наступает при встрече закрывающего маркера:

Listing 11: Обработка закрывающего маркера вызова функции

```

1  case ViewTerm.AngClose ignored -> {
2      bumpStep();
3      Expr<Term> argsExpr = Expr.empty();
4      String fn = null;
5      while (true) {
6          ViewTerm top = left.pollLast();
7          if (top instanceof ViewTerm.AngOpen ao) { fn = ao.fn(); break; }
8          if (top instanceof ViewTerm.Passive pt)
9              argsExpr = argsExpr.pushFront(pt.term());
10         else if (top instanceof ViewTerm.BulkPassive bp)
11             argsExpr = Expr.concat(bp.items(), argsExpr); // O(1)
12     }
13 }

```

При встрече `AngClose` мы снимаем элементы с хвоста `left` до парного `AngOpen` и попутно собираем выражение аргументов `argsExpr`. Одиночный `Passive` присоединяется через `pushFront` за $O(1)$, а `BulkPassive` — через `Expr.concat(bp.items(), argsExpr)`, тоже за $O(1)$ амортизированно. Обработка `ParenClose` устроена точно так же, только без поиска имени функции.

После сборки аргументов для встроенной функции результат сплющивается и помещается обратно в `right`; для пользовательской функции находится первое подходящее правило и его правая часть, с подставленными связываниями переменных, укладывается на `right` методом `pushResultToStack`.

Итеративный цикл вместо рекурсивного обхода даёт нам независимость от размера стека JVM: сколь угодно глубоко вложенная рефал-программа обрабатывается в константной глубине Java-стека, а реальный расход памяти зависит только от длины поля зрения.

1.3.3 Оптимизация `BulkPassive`

Без оптимизации подстановка значения переменной в правой части правила требовала поэлементной укладки: каждый `Term` из связанного значения помещался на стек `right` как отдельный объект `Passive`:

Listing 12: Наивная укладка переменной (до оптимизации)

```
1 case refal.ast.Var v -> {
2     Expr<Term> val = bindings.get(v.key());
3     // O(N) итераций: каждый терм становится Passive
4     for (Term t : val) {
5         right.addFirst(new ViewTerm.Passive(t));
6     }
7 }
```

При следующей встрече `AngClose` те же N элементов снимались с `left` по одному — ещё $O(N)$ работы на каждый шаг нормализации. В программе, которая на каждом шаге накапливает выражение длиной N (как `Loop/Select` из аналитической части), это давало $O(N^2)$ операций суммарно.

Оптимизация сводится к одному изменению: вместо перебора элементов переменной её значение помещается на стек одним объектом `BulkPassive`:

Listing 13: Укладка переменной через BulkPassive ($O(1)$)

```
1  case refal.ast.Var v -> {  
2      Expr<Term> val = bindings.get(v.key());  
3      if (!val.isEmpty()) {  
4          right.addFirst(new ViewTerm.BulkPassive(val)); //  $O(1)$   
5      }  
6  }
```

Когда обработчик `AngClose` снимает этот объект с `left`, он подмешивает его в накапливаемые аргументы вызовом `Expr.concat` — тоже амортизированно $O(1)$ (листинг (листинг 11)).

Важный инвариант: `BulkPassive` попадает в поле зрения ровно в той позиции, где раньше лежали бы отдельные `Passive` с тем же набором термов в том же порядке. Это значит, что замена не меняет порядок вычисления и не нарушает семантику Рефала — мы просто переносим ту же подпоследовательность одним объектом вместо N .

Эффект от оптимизации хорошо виден на двух тестах. Тест `03-concat.ref`, который раньше упирался в лимит в 100 миллионов шагов нормализации и не завершался, теперь проходит примерно за 10 секунд. На бенчмарке `Loop/Select` при $N = 20\,000$ наш интерпретатор показывает порядка 300 мс против секунды и более у `Refal-5 PZ` и `Refal-5J`.

2 Аналитическая часть

2.1 Стратегия тестирования

Тесты разбиты на три уровня по характеру проверяемого поведения.

Юнит-тесты проверяют отдельные компоненты в изоляции: раскрученную двустороннюю очередь (`ShallowDeque`, `DeepDeque`, `ExprOps.concat`), лексер, парсер, семантический анализ, сопоставитель шаблонов, конструктор результата и встроенные функции.

Интеграционные тесты прогоняют полную цепочку «разбор → семантика → нормализация» на подготовленных фикстурах: фиксируется конкретный вывод интерпретатора и коды возврата CLI.

Совместимостные тесты (`AutotestsRunnerTest`) запускают интерпретатор как внешний процесс на наборах тестовых программ, унаследованных от существующих реализаций Рефала, и проверяют результаты построчно.

В качестве фреймворка используется JUnit 5 [2]; все сравнения — через AssertJ [3]; покрытие кода — JaCoCo [4].

2.2 Состав тестов

Распределение тестов по классам приведено в таблице 4. Числа по классам `ShallowDequeTest`–`MainTest` получены прогоном `mvn test`; строка `AutotestsRunnerTest` — количество внешних .ref-программ, на которых запускался интерпретатор.

2.2.1 Юнит-тесты раскрученной очереди

`ShallowDequeTest` проверяет: пустой буфер; заполнение до предела через `pushFrontTight/pushBackTight`; операции разделения `separateFront/separateBack`; защиту от переполнения (`IllegalStateException` при вызове `tight`-метода на полном буфере); переход в `DeepDeque` через публичный `pushBack`; корректность `equals/hashCode` для буферов с одинаковым содержимым, но разной позицией `head` в массиве.

Таблица 4 — Распределение тестов по классам

Тестовый класс	Уровень	Кол-во
refal.deque.ShallowDequeTest	юнит	11
refal.deque.DeepDequeTest	юнит	9
refal.deque.ConcatTest	юнит	9
refal.lexer.LexerTest	юнит	12
refal.parser.ParserTest	юнит	30
refal.semantic.SemanticTest	юнит	12
refal.runtime.MatcherTest	юнит	14
refal.runtime.SubstitutorTest	юнит	7
refal.runtime.BuiltinsTest	юнит	10
refal.ast.AstTest	юнит	7
refal.runtime.RuntimeIntegrationTest	интеграционный	12
refal.MainTest	интеграционный	10
refal.AutotestsRunnerTest	совместимостный	81
Итого		224

DeepDequeTest проверяет построение глубокой очереди через расщепление мелкой; многократное переполнение, вызывающее рекурсивный рост поля mid; перенормировку mkDeep при уменьшении суммарного размера ниже CAPACITY; обход через iterator() в правильном порядке.

ConcatTest параметризован по четырём сценариям ExprOps.concat (SS, SD, DS, DD) и проверяет содержимое результата, его размер и тип (ShallowDeque при суммарном размере ≤ 9 , иначе DeepDeque).

2.2.2 Юнит-тесты лексера, парсера и семантики

LexerTest покрывает все классы токенов: ключевые слова, операторы, идентификаторы, переменные, строковые литералы в одинарных и двойных кавычках, целые числа со знаком, комментарии; учёт координат при многострочном вводе; исключения LexException при некорректных входах.

ParserTest содержит наибольшее число тестов (30): положительные случаи корректной грамматики; отрицательные случаи с ожидаемыми исключениями ParseException; параметризованные прогоны по фикстурам каталогов ok/ и bad-syntax/.

`SemanticTest` проверяет все виды семантических ошибок (`SemanticException.Kind`): повторное объявление функции, обращение к необъявленной функции, несоответствие типов переменной и др.

2.2.3 Юнит-тесты исполнения

`MatcherTest` (14 тестов) проверяет все классы переменных (`s`, `t`, `e`), повторные вхождения, перебор `e`-переменной с отказом и успехом, сопоставление со скобочными термами.

`SubstitutorTest` (7 тестов) проверяет правильное построение результата для всех видов узлов AST. Отдельно тестируется корректность при двукратном вхождении одной `e`-переменной в результат.

`BuiltinsTest` (10 тестов) покрывает каждую встроенную функцию: арифметику на макроцифрах, `Compare`, `Lower/Upper`, `Symb/Numb`, `Mu`.

2.2.4 Интеграционные тесты

`RuntimeIntegrationTest` (12 тестов) прогоняет интерпретатор на отобранных фикстурах из `src/test/resources/fixtures/ok/` и `fixtures/desugared/`: каждая фикстура — пара «программа.ref + ожидаемый вывод». Это сквозная проверка всей цепочки преобразований.

`MainTest` (10 тестов) проверяет CLI-обёртку: выбор точки входа, флаги `-entry/-expr`, корректные коды возврата (0 / 1 / 2 / код от `<Exit n>`), захват вывода в `stdout`.

2.2.5 Совместимостные тесты (`AutotestsRunnerTest`)

`AutotestsRunnerTest` запускает интерпретатор как внешний процесс на двух наборах тестовых программ.

Набор `refal-5j` (27 программ): 5 помечены как `MUST_PASS`; остальные используют синтаксис `:` (уточнение типа термина) или встроенные функции, не входящие в реализованное подмножество (`Random`, `GetEnv` и др.), поэтому ожидаемо не проходят.

Набор `desugared` (50 программ): 18 помечены как `MUST_PASS`; ещё 11 программ проходят, но не обязательны (тестируют `built-in` вне нашего жёсткого яд-

ра). Остальные требуют нереализованных встроенных (`SizeOf`, `GetEnv`, `Random`, `Time`, `Br/Dg` для файлового ввода/вывода), либо опираются на чтение существующих файлов.

2.3 Покрытие кода

Отчёт JaCoCo генерируется командой `mvn verify` в каталог `target/site/jacoco/`. Покрытие по основным пакетам по инструкциям байт-кода и веткам приведено в таблице 5.

Таблица 5 — Покрытие кода по пакетам (JaCoCo)

Пакет	Инструкции	Ветки
<code>refal.deque</code>	86,7 %	72,9 %
<code>refal.lexer</code>	90,3 %	80,9 %
<code>refal.parser</code>	86,5 %	76,1 %
<code>refal.semantic</code>	94,8 %	95,5 %
<code>refal.runtime</code>	85,6 %	74,7 %
Итого по проекту	87,3 %	76,9 %

В `pom.xml` закреплены минимальные пороги покрытия: $\geq 70\%$ по инструкциям и $\geq 55\%$ по веткам для пакетов `refal.deque` и `refal.runtime`. Сборка `mvn verify` падает, если порог нарушен; это страхует основные части реализации от регрессий.

Непокрытыми остаются защитные `IllegalStateException` в инвариантах структуры данных — они не достижимы при корректных входах, — а также ветви `equals/hashCode`, которые срабатывают только на маловероятных комбинациях позиций `head`.

2.4 Сравнительные прогоны

2.4.1 Условия измерений

Все прогоны выполнены в условиях, приведённых в таблице 6. Время фиксируется как полное время жизни внешнего процесса: запуск JVM, загрузка классов и собственно вычисление.

Таблица 6 — Условия сравнительных прогонов

Параметр	Значение
ЦПУ	Intel Core i5-11400H, 6 ядер / 12 потоков, 2,70 ГГц
ОЗУ	8 ГБ DDR4-3200
ОС	Windows 10 Pro 22H2 (сборка 19045)
JDK	Oracle JDK 25.0.3 LTS (HotSpot 25.0.3+9)
Refal-5 PZ	Version-PZ, релиз от 09.09.2025
Refal-5J	сборка от 13.03.2026

2.4.2 Тестовая программа Loop/Select

Тестовая программа (исходный код приведён в приложении А) выполняет N итераций; каждая строит пару скобочных термов — $(e.\text{Acc } A)$ или $(e.\text{Acc } B)$ — из растущего аккумулятора $e.\text{Acc}$. Функция `Select` в зависимости от текущего направления (`Left/Right`) возвращает одну из двух альтернатив; оставшийся терм следующего уровня присоединяется к аккумулятору. На каждом шаге аккумулятора $e.\text{Acc}$ встречается в обоих аргументах, а значит, при построении результата его значение нужно присоединить к новому выражению.

Именно на этой конкатенации структуры данных расходятся. При связном списке $e.\text{Acc}$ на k -й итерации имеет длину k , и каждый новый скобочный терм требует копирования всей цепочки: суммарно $1 + 2 + \dots + N = O(N^2)$. Раскрученная двусторонняя очередь выполняет ту же конкатенацию амортизированно за $O(1)$: значение переменной кладётся в стек нормализатора одной операцией `BulkPassive`, а сборка аргументов уже использует `Expr.concat` по схеме Окасаки [1]. Итоговая сложность цикла — $O(N)$.

2.4.3 Методика измерений

Для каждой реализации выполняется один прогревочный запуск (нужен для прогрева JIT-компилятора в Java-реализациях) и три измеряемых; в отчёт идёт медиана трёх измерений. Тайм-аут на один запуск — 120 секунд.

Размеры входа N покрывают два с половиной порядка: 100, 300, 600, 1000, 2000, 3000, 5000, 7000, 10 000, 15 000, 20 000.

Запуск выполняется скриптом `bench/loop-select-bench.ps1`:

Listing 14: Запуск бенчмарка Loop/Select

```
1 powershell -File bench\loop-select-bench.ps1 -WithPZ -WithR5J
```

2.4.4 Результаты

Медианные времена выполнения приведены в таблице 7. Прочерк означает, что реализация не была доступна для прогона.

Таблица 7 — Время выполнения Loop/Select (мс, медиана 3 прогонов)

N	refal5j-impl	Refal-5 PZ	Refal-5J
100	122	13	96
300	132	13	88
600	144	15	95
1 000	156	17	103
2 000	186	27	107
3 000	211	41	133
5 000	224	80	186
7 000	236	142	274
10 000	254	277	440
15 000	297	632	830
20 000	302	1059	1348

Те же данные представлены графически на рисунке 1: у нашей реализации зависимость времени от N ложится на прямую, у Refal-5 PZ и Refal-5J она растёт как парабола.

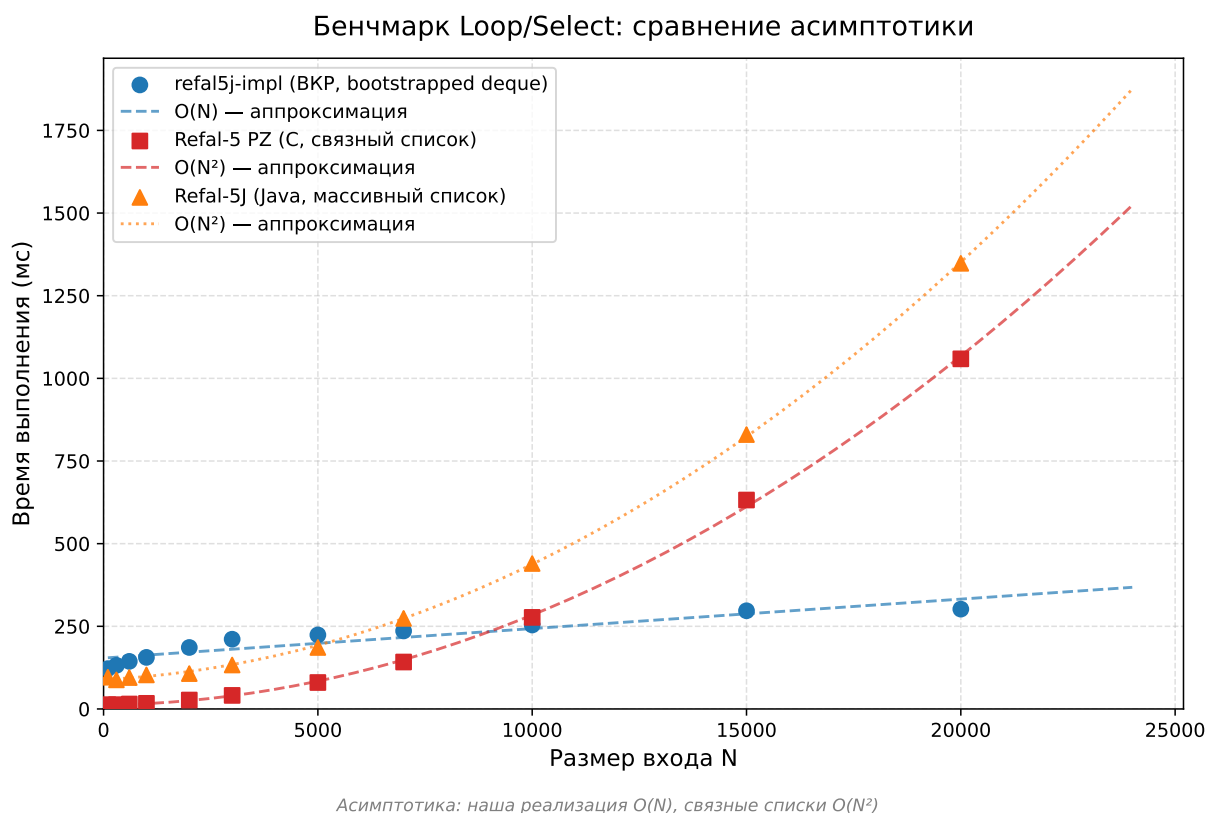


Рисунок 1 — Зависимость времени выполнения Loop/Select от размера входа N

2.4.5 Обсуждение результатов

На малых N наша реализация проигрывает обоим конкурентам. Причина простая: запуск JVM и прогрев JIT-компилятора стабильно стоят около 120 мс, и эта плата не зависит от размера входа. Refal-5 PZ — нативный бинарник, его старт укладывается в единицы миллисекунд; Refal-5J работает на той же JVM, что и мы, но тратит на инициализацию порядка 85–90 мс.

При $N \approx 7000$ кривая Refal-5J уже догоняет нашу, при $N \approx 9000$ её переоценивает Refal-5 PZ. После этой точки квадратичный рост конкурентов берёт своё: на $N = 10\,000$ PZ уже медленнее (277 мс против 254 мс), а на $N = 20\,000$ разрыв с PZ становится 3,5-кратным (1059 мс против 302 мс) и 4,5-кратным с Refal-5J (1348 мс против 302 мс).

Объяснение различия лежит в способе хранения объектного выражения. Refal-5 PZ и Refal-5J держат выражение в виде двусвязного списка термов: при построении нового скобочного терма (е. Асс A/B) на k -й итерации нужно скопировать всю накопленную часть длиной k . N итераций по $O(k)$ дают $O(N^2)$. Наш интерпретатор устроен иначе: значение переменной идёт на стек одним объек-

том BulkPassive, а сборка аргументов вызова использует `Expr.concat` — по схеме Окасаки это амортизированная константа независимо от длины операндов. N итераций по $O(1)$ дают $O(N)$ — ровно то, что показывает прямая на графике.

Стоит честно оговорить границы применимости этого замера. Во-первых, мы измеряем время жизни всего процесса, поэтому в наши ≈ 120 мс на $N = 100$ входит почти исключительно запуск JVM, а не работа интерпретатора. Во-вторых, диапазон N до 20 000 показывает только начальный участок параболы конкурентов; на больших N разрыв будет расти быстрее, но прогон при $N \sim 10^5$ упирается в наш тайм-аут 120 с. В-третьих, мы сравниваем интерпретатор с интерпретатором (не с компиляторами Refal-5 PZ и Refal-5J в нативный код), чтобы сопоставление имело смысл.

ЗАКЛЮЧЕНИЕ

В ходе практики реализовано базисное подмножество Рефала-5 на Java 25. Объектное выражение представлено иммутабельной раскрученной двусторонней очередью `Expr<T>` с двумя реализациями `ShallowDeque` и `DeepDeque`; цикл нормализации построен итеративно на двух стеках и не зависит от глубины рекурсии исходной программы. Отдельно реализована оптимизация `BulkPassive`: значение переменной попадает на стек одной операцией, без поэлементного копирования, что превращает типовой шаг с конкатенацией e -переменной из $O(k)$ в $O(1)$ амортизировано.

Тестовый каркас включает 143 JUnit-теста на компоненты и 79 внешних программ, унаследованных от существующих реализаций Рефала (`refal-5j` и `refal-5-framework`). По обязательной части наборов проходит 5/5 автотестов и 18/18 десахаренных тестов; оставшиеся отказы вызваны либо расширенным синтаксисом Рефала-5 (условия, блоки, уточнение типа через двоеточие), либо нереализованными в базисном подмножестве встроенными функциями (`Time`, `SizeOf`, `Random`, `GetEnv` и др.).

Сравнительный замер на программе `Loop/Select` подтвердил ожидание: у нашей реализации время линейно по N , у `Refal-5 PZ` и `Refal-5J` квадратично. На $N = 20\,000$ наша реализация работает в 3,5 раза быстрее `Refal-5 PZ` и в 4,5 раза быстрее `Refal-5J` — несмотря на накладные расходы запуска JVM, которые играют единичным миллисекундам нативного бинарника `PZ` на малых входах.

Дальнейшее развитие реализации укладывается в две линии. Первая — расширить набор встроенных функций до полного интерфейса Рефала-5, чтобы покрыть оставшиеся десахаренные тесты (файловый ввод/вывод, `Time`, `SizeOf`, `Random`, системные функции). Вторая — добавить поддержку расширенного синтаксиса Рефала-5 (условия, блоки, уточнение типа), что позволит обходиться без внешнего рессахаривателя и запускать автотесты в их исходном виде.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Okasaki C. Purely Functional Data Structures. — Cambridge University Press, 1998.
2. JUnit 5 User Guide [Электронный ресурс]. — URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обр. 05.05.2026).
3. AssertJ – fluent assertions for Java [Электронный ресурс]. — URL: <https://assertj.github.io/doc/> (дата обр. 05.05.2026).
4. JaCoCo Java Code Coverage Library [Электронный ресурс]. — URL: <https://www.jacoco.org/jacoco/trunk/doc/> (дата обр. 05.05.2026).

А Исходный код бенчмарка Loop/Select

Листинг 1: Бенчмарк Loop/Select (bench/loop-select.ref)

```
1 * Loop/Select benchmark - demonstrates  $O(N)$  for
   bootstrapped deque
2 * Run: java -jar target/refal5q.jar bench/loop-select.ref
   10000
3
4 $ENTRY Go {
5   = <TimeElapsed 0>
6     <Loop Left () <Numb <Arg 1>>>
7     <Prout <TimeElapsed>>;
8 }
9
10 Loop {
11   s.Side (e.Acc) 0 = /* done */;
12   s.Side (e.Acc) s.Rest
13     = <Loop <Select s.Side (e.Acc A) (e.Acc B)> <- s.Rest
       1>>;
14 }
15
16 Select {
17   Left  t.Left t.Right = Right t.Left;
18   Right t.Left t.Right = Left  t.Right;
19 }
```

Б Инструкция по воспроизведению измерений

Предварительные требования

- Java 25 LTS и Apache Maven 3.9.x (должны быть доступны в PATH);
- PowerShell 5.1 или новее;
- для сравнения с Refal-5 PZ: дистрибутив Refal-5 PZ с исполняемыми `refc.exe` и `refgo.exe`;
- для сравнения с Refal-5J: собранный `r5jc.jar` и `r5jrt.jar`.

Сборка интерпретатора

Listing 15: Сборка и запуск бенчмарка

```
1 cd refal5j-impl
2 mvn package -DskipTests
3
4 # our implementation only:
5 powershell -File bench\loop-select-bench.ps1
6
7 # with Refal-5 PZ:
8 powershell -File bench\loop-select-bench.ps1 -WithPZ
9
10 # with all three:
11 powershell -File bench\loop-select-bench.ps1 -WithPZ -WithR5J
```

Результаты сохраняются в `bench/loop-select-results.csv`.

Построение графика

Listing 16: Построение графика

```
1 pip install matplotlib numpy
2 python bench\plot-benchmark.py --out bench\loop-select-graph.pdf
```

Настройка путей

Пути к исполняемым файлам Refal-5 PZ и Refal-5J задаются переменными \$Refal5, \$R5JBin и \$R5JLib в начале файла `bench/loop-select-bench.ps1`.

В Реализация функций конкатенации

Четыре вспомогательных метода класса `ExprOps`, на которые опирается `Expr.concat`. Каждый обрабатывает свою комбинацию типов операндов; диспетчер показан в листинге 9 основного текста.

Listing 17: `concatSS`: оба операнда `ShallowDeque`

```
1 private static <T> Expr<T> concatSS(ShallowDeque<T> a, ShallowDeque<T> b)
   ↪ {
2     int total = a.size() + b.size();
3     if (total <= CAPACITY) {
4         ShallowDeque<T> r = a;
5         for (T x : iter(b)) {
6             r = r.pushBackTight(x);
7         }
8         return r;
9     }
10    // total > CAPACITY: один из буферов полный или почти полный;
11    // делаем новый Deep с пустой mid.
12    return DeepDeque.mkDeep(a, ShallowDeque.empty(), b, total);
13 }
```

Listing 18: `concatSD`: `ShallowDeque` + `DeepDeque`

```
1 private static <T> Expr<T> concatSD(ShallowDeque<T> a, DeepDeque<T> b) {
2     ShallowDeque<T> bf = b.front();
3     int total = b.size() + a.size();
4     // Сливаем 'a' слева в b.front. Если итог влезает в один Shallow,
5     // он становится новым front; иначе расщепляем на два,
6     // второй идёт в начало mid.
7     int bfTotal = a.size() + bf.size();
8     if (bfTotal <= CAPACITY) {
9         ShallowDeque<T> mergedFront = ShallowDeque.empty();
10        for (T x : iter(a)) mergedFront = mergedFront.pushBackTight(x);
11        for (T x : iter(bf)) mergedFront = mergedFront.pushBackTight(x);
12        return DeepDeque.mkDeep(mergedFront, b.mid(), b.back(), total);
13    }
14    // Иначе формируем плоский массив и делим примерно пополам.
```

```

15     Object[] tmp = new Object[bfTotal];
16     int i = 0;
17     for (T x : iter(a)) tmp[i++] = x;
18     for (T x : iter(bf)) tmp[i++] = x;
19     int leftLen = bfTotal - CAPACITY;
20     if (leftLen < ShallowDeque.MIN_FILL) leftLen = ShallowDeque.MIN_FILL;
21     int rightLen = bfTotal - leftLen;
22     ShallowDeque<T> head = ShallowDeque.ofArray(tmp, 0, leftLen);
23     ShallowDeque<T> tail = ShallowDeque.ofArray(tmp, leftLen, rightLen);
24     // head становится новым front; tail - первым бакетом mid.
25     Expr<ShallowDeque<T>> newMid = b.mid().pushFront(tail);
26     return DeepDeque.mkDeep(head, newMid, b.back(), total);
27 }

```

Listing 19: concatDS: DeepDeque + ShallowDeque

```

1 private static <T> Expr<T> concatDS(DeepDeque<T> a, ShallowDeque<T> b) {
2     ShallowDeque<T> ab = a.back();
3     int total = a.size() + b.size();
4     int abTotal = ab.size() + b.size();
5     if (abTotal <= CAPACITY) {
6         ShallowDeque<T> mergedBack = ShallowDeque.empty();
7         for (T x : iter(ab)) mergedBack = mergedBack.pushBackTight(x);
8         for (T x : iter(b)) mergedBack = mergedBack.pushBackTight(x);
9         return DeepDeque.mkDeep(a.front(), a.mid(), mergedBack, total);
10    }
11    Object[] tmp = new Object[abTotal];
12    int i = 0;
13    for (T x : iter(ab)) tmp[i++] = x;
14    for (T x : iter(b)) tmp[i++] = x;
15    int rightLen = abTotal - CAPACITY;
16    if (rightLen < ShallowDeque.MIN_FILL) rightLen =
17        ↪ ShallowDeque.MIN_FILL;
18    int leftLen = abTotal - rightLen;
19    ShallowDeque<T> head = ShallowDeque.ofArray(tmp, 0, leftLen);
20    ShallowDeque<T> tail = ShallowDeque.ofArray(tmp, leftLen, rightLen);
21    Expr<ShallowDeque<T>> newMid = a.mid().pushBack(head);
22    return DeepDeque.mkDeep(a.front(), newMid, tail, total);
23 }

```

Listing 20: concatDD: два DeepDeque — ключевой случай $O(1)$ амортиз.

```
1 private static <T> Expr<T> concatDD(DeepDeque<T> a, DeepDeque<T> b) {
2     int total = a.size() + b.size();
3     ShallowDeque<T> ab = a.back();
4     ShallowDeque<T> bf = b.front();
5     int boundary = ab.size() + bf.size();
6
7     Expr<ShallowDeque<T>> aMidPlus = a.mid();
8     if (boundary == 0) {
9         // оба пограничных пусты - ничего не добавляем.
10    } else if (boundary <= CAPACITY) {
11        ShallowDeque<T> merged = ShallowDeque.empty();
12        for (T x : iter(ab)) merged = merged.pushBackTight(x);
13        for (T x : iter(bf)) merged = merged.pushBackTight(x);
14        aMidPlus = aMidPlus.pushBack(merged);
15    } else {
16        Object[] tmp = new Object[boundary];
17        int i = 0;
18        for (T x : iter(ab)) tmp[i++] = x;
19        for (T x : iter(bf)) tmp[i++] = x;
20        int rightLen = boundary - CAPACITY;
21        if (rightLen < ShallowDeque.MIN_FILL) rightLen =
22            ↪ ShallowDeque.MIN_FILL;
23        int leftLen = boundary - rightLen;
24        ShallowDeque<T> m1 = ShallowDeque.ofArray(tmp, 0, leftLen);
25        ShallowDeque<T> m2 = ShallowDeque.ofArray(tmp, leftLen,
26            ↪ rightLen);
27        aMidPlus = aMidPlus.pushBack(m1).pushBack(m2);
28    }
29
30    // Рекурсивное склеивание middle-очередей: O(1) амортиз. по Окасаки.
31    Expr<ShallowDeque<T>> newMid = concat(aMidPlus, b.mid());
32    return DeepDeque.mkDeep(a.front(), newMid, b.back(), total);
33 }
```